

hw1-definitions-and-notation

CSE20W26

Due: 01/15/26 at 5pm (no penalty late submission until 8am next morning)

In this assignment,

You will practice reading and applying definitions to get comfortable working with mathematical language.

Relevant class material: Week 1.

You will submit this assignment via Gradescope (<https://www.gradescope.com>) in the assignment called “hw1-definitions-and-notation”.

For all HW assignments: These homework assignments may be done individually or in groups of up to four students. Please ensure your name(s) and PID(s) are clearly visible on the first page of your homework submission, start each question on a new page, and upload the PDF to Gradescope. If you’re working in a group, *submit only one submission per group*: one partner uploads the submission through their Gradescope account and then adds the other group member(s) to the Gradescope submission by selecting their name(s) in the “Add Group Members” dialog box. You will need to re-add your group member(s) every time you resubmit a new version of your assignment.

All submitted homework for this class must be typed. You can use a word processing editor if you like (Microsoft Word, Open Office, Notepad, Vim, Google Docs, etc.) but you might find it useful to take this opportunity to learn LaTeX. LaTeX is a markup language used widely in computer science and mathematics. The homework assignments are typed using LaTeX and you can use the source files as templates for typesetting your solutions.

Integrity reminders

- Problems should be solved together, not divided up between the partners. The homework is designed to give you practice with the main concepts and techniques of the course, while getting to know and learn from your classmates.
- Use only resources explicitly allowed for each assignment. Resources not affiliated with this quarter’s version of the class may use inconsistent notation or definitions. When consulting

resources, balance getting the help you need while also protecting the learning experience you will have when the ‘aha’ moments of solving the problem authentically happen.

- Do not share written solutions or partial solutions for homework with other students in the class who are not in your group. Doing so would dilute their learning experience and detract from their success in the class.

When evaluating your homework solutions, we will be considering:

- The correctness of any final answers.
- Whether ideas are presented clearly and logically. You should explain how you arrived at your conclusions, using mathematically sound reasoning. Whether you use formal proof techniques or write a more informal argument for why something is true, your answers should always be well-supported. Your goal should be to convince the reader that your results and methods are sound. A rule of thumb for the level of detail to include is to imagine someone who is taking the class alongside you but missed about a week’s worth of content: does your writeup have enough detail for them to learn from it?

To demonstrate your honest effort in answering homework questions, we expect you to include your attempt to answer *each* part of the question. If you get stuck with your attempt, you can still demonstrate your effort by explaining where you got stuck and what you did to try to get unstuck (e.g. reviewing annotated notes from class, checking the relevant textbook sections, looking up comparable questions on the relevant weekly review quizzes, asking questions on Piazza, attending (instructor/TA/tutor) office hours, consulting generative AI tools, etc.).

Assigned questions

1. Modeling

- (a) In class, we used 4-tuples to represent the user ratings for four movies. This representation is memory-efficient because we use the order of the components in the 4-tuples to represent which move is being rated. However, it is not easily extended when we want to add new movies to the database. Define a new model that would allow us to represent the user ratings of movie databases where we allow for new movies to be added. Use only the data types we have talked about in class: sets, n -tuples, and strings. Explain the design choices that you used to define your model by referencing properties of the data-type(s) you choose. Demonstrate your model by showing how the rating of the user who dislikes Superman, Wicked and likes KPop Demon Hunters, A Minecraft Movie is represented.
- (b) Colors can be described as amounts of red, green, and blue mixed together ¹ Mathematically, a color can be represented as a 3-tuple (r, g, b) where r represents the red component, g the green

¹This RGB representation is common in web applications. Many online tools are available to play around with mixing these colors, e.g. https://www.w3schools.com/colors/colors_rgb.asp.

component, b the blue component and where each of r , g , b must be a value from this collection of numbers:

{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 16, 17, 18, 19, 20, 21, 22, 23, 24, 25, 26, 27, 28, 29, 30, 31, 32, 33, 34, 35, 36, 37, 38, 39, 40, 41, 42, 43, 44, 45, 46, 47, 48, 49, 50, 51, 52, 53, 54, 55, 56, 57, 58, 59, 60, 61, 62, 63, 64, 65, 66, 67, 68, 69, 70, 71, 72, 73, 74, 75, 76, 77, 78, 79, 80, 81, 82, 83, 84, 85, 86, 87, 88, 89, 90, 91, 92, 93, 94, 95, 96, 97, 98, 99, 100, 101, 102, 103, 104, 105, 106, 107, 108, 109, 110, 111, 112, 113, 114, 115, 116, 117, 118, 119, 120, 121, 122, 123, 124, 125, 126, 127, 128, 129, 130, 131, 132, 133, 134, 135, 136, 137, 138, 139, 140, 141, 142, 143, 144, 145, 146, 147, 148, 149, 150, 151, 152, 153, 154, 155, 156, 157, 158, 159, 160, 161, 162, 163, 164, 165, 166, 167, 168, 169, 170, 171, 172, 173, 174, 175, 176, 177, 178, 179, 180, 181, 182, 183, 184, 185, 186, 187, 188, 189, 190, 191, 192, 193, 194, 195, 196, 197, 198, 199, 200, 201, 202, 203, 204, 205, 206, 207, 208, 209, 210, 211, 212, 213, 214, 215, 216, 217, 218, 219, 220, 221, 222, 223, 224, 225, 226, 227, 228, 229, 230, 231, 232, 233, 234, 235, 236, 237, 238, 239, 240, 241, 242, 243, 244, 245, 246, 247, 248, 249, 250, 251, 252, 253, 254, 255}

(This is the same definition as in the Week 1 Review quiz.)

Can you find two different 3-tuples that represent colors that are indistinguishable to your eye? You can use the website in the footnote to play around with different choices of red, green, and blue levels to see if you can distinguish between the resulting colors. Why or why not?

A complete answer will include the specific example 3-tuples that work, along with a description of the colors that they represent and why they are indistinguishable, or an explanation of why there can't be such an example.

2. Sets and functions

- (a) Each of the sets below is described using set builder notation or recursion or as a result of set operations applied to other known sets. Rewrite each of the sets using the roster method.

Remember our discussions of data-types: use clear notation that is consistent with our class notes and definitions to communicate the data-types of the elements in each set.

Sample response that can be used as reference for the detail expected in your answer:

The set $\{A\} \circ \{AU, AC, AG\}$ can be written using the roster method as

$$\{AAU, AAC, AAG\}$$

because set-wise concatenation gives a set whose elements are all possible results of concatenating an element of the left set with an element of the right set. Since the left set in this example only has one element, namely A , each of the elements of the set we described starts with A . There are three elements of this set, one for each of the distinct elements of the right set.

i.

$$\{n \in \mathbb{Z}^+ \mid n \leq 3\} \times \{m \in \mathbb{N} \mid m \leq 3\}$$

ii. The set X defined recursively as

Basis Step: $1 \in X, 3 \in X, 5 \in X$

Recursive Step: If the integer $n \in X$, then the result of multiplying n by -1 is in X

iii.

$$\{x \in S \mid rnalen(x) = 2\} \circ \{x \in S \mid rnalen(x) = 0\}$$

where S is the set of RNA strands and $rnalen$ is the recursively defined function that we discussed in class,

$$\begin{array}{ll} rnalen : S & \rightarrow \mathbb{Z}^+ \\ \text{Basis Step:} & \text{If } b \in B \text{ then } rnalen(b) = 1 \\ \text{Recursive Step:} & \text{If } s \in S \text{ and } b \in B, \text{ then } rnalen(sb) = 1 + rnalen(s) \end{array}$$

iv.

$$\{(r, g, b) \in C \mid r + g + b = 2 \text{ and } g = 1\}$$

where $C = \{(r, g, b) \mid 0 \leq r \leq 255, 0 \leq g \leq 255, 0 \leq b \leq 255, r \in \mathbb{N}, g \in \mathbb{N}, b \in \mathbb{N}\}$.

(b) Recall the function which takes an ordered pair of ratings 4-tuples and returns a measure of the difference between them

$$d_0 : \{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4 \rightarrow \mathbb{R}$$

given by

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) = \sqrt{(x_1 - y_1)^2 + (x_2 - y_2)^2 + (x_3 - y_3)^2 + (x_4 - y_4)^2}$$

Define a **new** function which we'll call d_{new} with the same domain $\{-1, 0, 1\}^4 \times \{-1, 0, 1\}^4$ and codomain $\mathbb{R}$ but where there is some example pair of ratings 4-tuples

$$((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4))$$

where

$$d_0((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4)) \neq d_{new}((x_1, x_2, x_3, x_4), (y_1, y_2, y_3, y_4))$$

Your answer should include **both** a precise and clear definition for the rule defining d_{new} which unambiguously specifies output for each input of the function **and** the example ordered pair of ratings 4-tuples that demonstrate that the functions are not equal. Also include a justification of your answers with (clear, correct, complete) calculations for each of the function applications and/or references to definitions and connecting them with the desired conclusion.

- (c) A function *basecount* that computes the number of a given base b appearing in a RNA strand s is defined recursively:

$$\textit{basecount} : S \times B \rightarrow \mathbb{N}$$

Basis Step:

$$\text{If } b_1 \in B, b_2 \in B \quad \textit{basecount}((b_1, b_2)) = \begin{cases} 1 & \text{when } b_1 = b_2 \\ 0 & \text{when } b_1 \neq b_2 \end{cases}$$

Recursive Step:

$$\text{If } s \in S, b_1 \in B, b_2 \in B \quad \textit{basecount}((sb_1, b_2)) = \begin{cases} 1 + \textit{basecount}((s, b_2)) & \text{when } b_1 = b_2 \\ \textit{basecount}((s, b_2)) & \text{when } b_1 \neq b_2 \end{cases}$$

Consider the function application

$$\textit{basecount}((\text{ACAU}, \text{A}))$$

What is the input? What is the output? Give an example of a different choice of input that gives the same output.

Your answer should include clearly labeled answers to each of the three parts of the question, along with a justification for the values of the applications that makes specific reference to the parts of the recursive definition of the *basecount* function used to calculate it.